

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA

E.P. de Ingeniería de Telecomunicaciones



Entregable 2: Diseño Detallado + Simulación Completa

**Diseño e implementación de un modulador FSK digital basado en Arduino
con selección de secuencia binaria mediante DIP Switch y medición de
frecuencia usando un segundo Arduino**

DOCENTE:

Villafuerte Barreto, Hernán Oswaldo

CURSO:

Circuitos Digitales

INTEGRANTES:

Farro Pomajulca Leonardo Jose

Cruz Vargas José Antonio

Bellido Araujo Alvaro Daniel

Lima, 2026.

Diseño Detallado y Simulación Funcional de un Modulador FSK Basado en Arduino

Resumen— Este documento presenta el diseño detallado y la simulación funcional de un sistema modulador por desplazamiento de frecuencia (FSK, Frequency Shift Keying) implementado sobre dos placas Arduino UNO. El primer Arduino lee una secuencia de 4 bits proveniente de un DIP switch, la interpreta mediante un decodificador que direcciona una memoria ROM de dos frecuencias (1200 Hz para el bit 0 y 2200 Hz para el bit 1), y controla el proceso mediante una máquina de estados finitos (FSM) sincronizada por un temporizador de software. La señal resultante se suaviza parcialmente con un filtro RC y se transmite hacia un segundo Arduino, que actúa como instrumento de medición de frecuencia mediante captura de pulsos. Se presentan el diagrama de bloques, el diagrama ASM, el mapa de memoria/registros, el código estructurado comentado, el plan de pruebas con su justificación de cobertura funcional, y el análisis teórico de las métricas involucradas (frecuencia, periodo de bit, tasa de transmisión y respuesta del filtro RC).

Términos clave— FSK, Arduino, máquina de estados finitos, memoria ROM, decodificador, modulación digital, filtro RC.

I. INTRODUCCIÓN

La modulación por desplazamiento de frecuencia (FSK) es una técnica de comunicación digital en la que la información binaria se transmite variando la frecuencia de una portadora entre dos valores discretos, en lugar de representar los bits como niveles de voltaje alto o bajo. Este principio es la base de sistemas históricos como los módems telefónicos, y se mantiene vigente en telemetría, radioenlaces simples y sistemas de alarma.

El presente entregable corresponde a la fase de diseño detallado y simulación funcional completa del proyecto de

II. ARQUITECTURA DEL SISTEMA

La arquitectura general se organiza en cinco bloques funcionales conectados en cascada, tal como se ilustra en la Fig. 1: (1) un registro de entrada que captura el estado del DIP switch de 4 posiciones mediante resistencias de pull-up internas; (2) un bloque decodificador que, junto con una memoria ROM de dos posiciones, traduce cada bit leído en su frecuencia correspondiente; (3) un registro/contador de índice que recorre las cuatro posiciones del vector de bits; (4) un temporizador de software basado en `millis()` que define la duración de cada bit transmitido (1000 ms); y (5) el generador de señal FSK, implementado con la función `tone()` sobre el pin digital D9.

La salida del Arduino 1 se suaviza mediante un filtro RC pasivo antes de ingresar al Arduino 2, el cual mide el periodo de la señal recibida en el pin D7 mediante la función

III. DISEÑO LÓGICO DETALLADO

A. Registro de entrada. El vector `bits[4]` almacena el estado invertido de los pines D2 a D5 (configurados con `INPUT_PULLUP`, por lo que un switch cerrado a tierra se lee como 0 lógico y se invierte con el operador `!` para obtener el bit real 1). Este vector se actualiza continuamente en cada

curso. Se documenta la arquitectura del sistema, el diagrama de transición de estados (ASM), el mapa de memoria del decodificador, el código estructurado, el plan de simulación con sus casos de prueba, el análisis teórico de las métricas esperadas y el guión de la demostración en laboratorio.

El sistema completo emplea dos placas Arduino UNO: el Arduino 1 cumple el rol de transmisor/modulador, mientras que el Arduino 2 cumple el rol de instrumento de medición, permitiendo verificar experimentalmente que las frecuencias generadas correspondan a los valores esperados de 1200 Hz y 2200 Hz.

`pulseIn()`, calcula la frecuencia resultante y la reporta por el puerto serial, cerrando así el lazo de verificación experimental del sistema.

Fig. 1. Diagrama de bloques del sistema modulador FSK

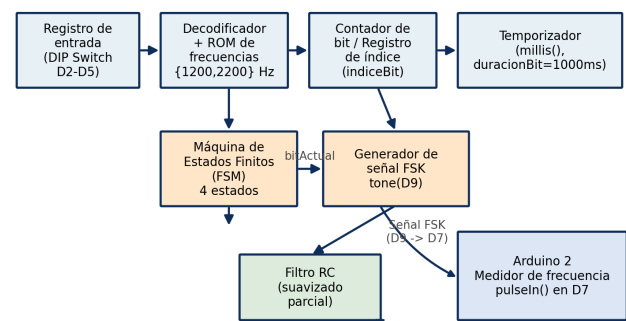


Fig. 1. Diagrama de bloques del sistema modulador FSK.

iteración del bucle principal, comportándose como un registro paralelo de 4 bits.

B. Decodificador y memoria ROM. El arreglo `frecuencias[2] = {1200, 2200}` constituye la memoria ROM del sistema. El bit leído en la posición actual (`bitActual`) actúa como dirección de un bit sobre esta memoria: la dirección 0 decodifica a 1200 Hz y la dirección 1 decodifica a 2200 Hz.

Esta operación de selección —una entrada binaria que activa una única salida entre dos posibles— corresponde exactamente a la función de un decodificador 1 a 2.

C. Registro de índice / contador. La variable indiceBit funciona como un contador módulo 4 que recorre las posiciones del vector bits[]. Al alcanzar el valor 4 se reinicia a 0, reiniciando el ciclo completo de transmisión de la trama de 4 bits.

D. Temporizador. El temporizador se implementa por software comparando millis() con la marca de tiempo tiempoAnterior. Cuando la diferencia supera duracionBit (1000 ms), se habilita la transición de estado, generando una base de tiempo estable sin bloquear el procesador (a diferencia de delay()).

E. Máquina de estados finitos. El sistema completo se modela como una FSM de cuatro estados (S0–S3), uno por cada bit de la trama. En cada estado la máquina: (i) lee el bit correspondiente, (ii) accede al decodificador/ROM para obtener la frecuencia, (iii) genera la señal con tone() y (iv)

espera la señal del temporizador para transitar al siguiente estado. La Fig. 2 muestra el diagrama ASM correspondiente.

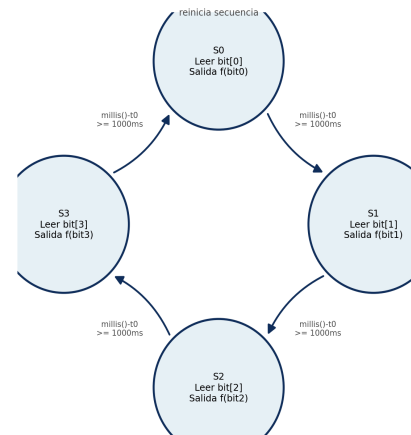


Fig. 2. Diagrama ASM: transición cíclica de 4 estados de lectura/transmisión

Fig. 2. Diagrama ASM de la máquina de estados finitos del modulador.

IV. DIAGRAMA DE TRANSICIÓN DE ESTADOS Y TABLA DE VERDAD

La Tabla I resume la tabla de transición de estados de la FSM, indicando para cada estado el bit leído, la dirección decodificada, la frecuencia de salida y la condición de transición al siguiente estado.

TABLA I. TABLA DE TRANSICIÓN DE ESTADOS (ASM)

Estado actual	Bit leído (indiceBit)	Dirección ROM	Frecuencia de salida	Condición → siguiente
S0	bits[0]	bitActual (0/1)	1200 Hz / 2200 Hz	millis()–t ₀ ≥ 1000 ms → S1
S1	bits[1]	bitActual (0/1)	1200 Hz / 2200 Hz	millis()–t ₀ ≥ 1000 ms → S2
S2	bits[2]	bitActual (0/1)	1200 Hz / 2200 Hz	millis()–t ₀ ≥ 1000 ms → S3
S3	bits[3]	bitActual (0/1)	1200 Hz / 2200 Hz	millis()–t ₀ ≥ 1000 ms → S0 (reinicio)

La Tabla II detalla la tabla de verdad del decodificador de frecuencia (ROM de 2 posiciones), núcleo de la operación de modulación.

TABLA II. TABLA DE VERDAD DEL DECODIFICADOR / ROM

Dirección (bitActual)	Valor binario del bit	Frecuencia decodificada (Hz)
0	0 lógico	1200
1	1 lógico	2200

V. MAPA DE MEMORIA Y MAPA DE REGISTROS

La Tabla III presenta el mapa de memoria/registros del sistema, incluyendo el rango de direcciones de la ROM de frecuencias, los registros de estado del programa y su función dentro del decodificador.

TABLA III. MAPA DE MEMORIA Y REGISTROS

Elemento	Rango / Dirección	Función	Tipo
frecuencias[]	0x0 – 0x1 (2 posiciones)	ROM: dirección de 1 bit → frecuencia	Memoria de solo lectura
bits[]	0x0 – 0x3 (4 posiciones)	Registro paralelo de entrada (DIP switch)	Registro
indiceBit	0 – 3 (contador módulo 4)	Puntero/dirección activa dentro de bits[]	Registro de control
pinesEntrada[]	D2, D3, D4, D5	Mapeo de bits físicos a entradas digitales	Registro de E/S
pinSalida	D9	Salida de la señal FSK modulada	Registro de E/S
pinMedicion (Arduino 2)	D7	Entrada de la señal FSK para medición	Registro de E/S

VI. CÓDIGO ESTRUCTURADO (PSEUDOCÓDIGO / FIRMWARE COMENTADO)

A continuación se presenta el firmware de ambos módulos, estructurado por bloques funcionales (registro de entrada, ROM/decodificador, temporizador y FSM en el Arduino 1; captura de pulsos en el Arduino 2), con comentarios que señalan la correspondencia entre cada fragmento y el concepto digital que implementa.

Listado 1. Arduino 1 — Modulador FSK.

```
// ===== Arduino 1: Modulador FSK con secuencia de 4 bits =====
// Entradas: DIP Switch en D2, D3, D4 y D5 usando INPUT_PULLUP
// Salida FSK: D9
const int pinesEntrada[4] = {2, 3, 4, 5};
const int pinSalida = 9;

// --- Memoria ROM de frecuencias ---
// dirección 0 -> 1200 Hz | dirección 1 -> 2200 Hz
const int frecuencias[2] = {1200, 2200};

int bits[4]; // Registro paralelo de entrada
int indiceBit = 0; // Contador / puntero de estado (S0..S3)
int bitActual = 0; // Bit decodificado en el estado actual
int frecuenciaActual = 0;
unsigned long tiempoAnterior = 0;
const unsigned long duracionBit = 1000; // Base de tiempo del temporizador (ms)

void setup() {
  Serial.begin(9600);
  for (int i = 0; i < 4; i++) {
    pinMode(pinesEntrada[i], INPUT_PULLUP); // Registro de entrada
  }
  pinMode(pinSalida, OUTPUT);
}

void loop() {
  // --- Actualizar registro de entrada (DIP switch) ---
  for (int i = 0; i < 4; i++) {
    bits[i] = !digitalRead(pinesEntrada[i]);
  }

  // --- Temporizador: habilita la transición de estado cada 1000 ms ---
  if (millis() - tiempoAnterior >= duracionBit) {
    tiempoAnterior = millis();
    bitActual = bits[indiceBit];
```

```

// --- Decodificador + acceso a ROM ---
frecuenciaActual = frecuencias[bitActual];

// --- Generación de la señal FSK en D9 ---
tone(pinSalida, frecuenciaActual);

Serial.print("Bit transmitido "); Serial.print(indiceBit + 1);
Serial.print(": "); Serial.print(bitActual);
Serial.print(" | Frecuencia: "); Serial.print(frecuenciaActual);
Serial.println(" Hz");

// --- Avance del contador de estado (FSM) ---
indiceBit++;
if (indiceBit >= 4) {
    indiceBit = 0;
    Serial.println("---- Reinicia secuencia ----");
}
}
}

```

Listado 2. Arduino 2 — Medidor de frecuencia.

```

// ===== Arduino 2: Medidor de frecuencia =====
// Entrada de señal: D7
const int pinMedicion = 7;

void setup() {
    pinMode(pinMedicion, INPUT);
    Serial.begin(9600);
    Serial.println("MEDIDOR LISTO");
}

void loop() {
    // Captura de los semiperiodos alto y bajo de la señal recibida
    unsigned long tiempoAlto = pulseIn(pinMedicion, HIGH, 100000);
    unsigned long tiempoBajo = pulseIn(pinMedicion, LOW, 100000);

    if (tiempoAlto == 0 || tiempoBajo == 0) {
        Serial.println("MEDIDOR | Sin senal detectada");
    } else {
        float periodo = tiempoAlto + tiempoBajo; // periodo en microsegundos
        float frecuencia = 1000000.0 / periodo; // f = 1 / T
        Serial.print("MEDIDOR | Frecuencia medida: ");
        Serial.print(frecuencia);
        Serial.println(" Hz");
    }
    delay(1000);
}

```

VII. PLAN DE SIMULACIÓN

A. Estímulos de entrada. El estímulo de entrada corresponde a las 4 posiciones del DIP switch, lo que define un espacio de $2^4 = 16$ combinaciones posibles. Para la validación funcional se seleccionaron combinaciones representativas: la secuencia 'todo ceros' (0000), la secuencia 'todo unos' (1111), la secuencia alternante 1010 y su complemento 0101, además de dos combinaciones mixtas adicionales (1100 y 0011) para verificar transiciones consecutivas de un mismo estado lógico.

B. Justificación de cobertura funcional. Las combinaciones elegidas garantizan: (i) cobertura de las dos ramas del decodificador (bit 0 y bit 1) en cada uno de los cuatro estados de la FSM; (ii) cobertura de las cuatro transiciones de estado $S0 \rightarrow S1 \rightarrow S2 \rightarrow S3 \rightarrow S0$; (iii) verificación de flancos de cambio de frecuencia (transición $1200 \leftrightarrow 2200$ Hz) en los casos alternantes; y (iv) verificación del caso sin transición de frecuencia (bits repetidos) en los casos 0000 y 1111. Adicionalmente, se contempla la desconexión de la señal para validar la rama de 'sin señal detectada' del Arduino 2 (tiempo de espera de pulseIn agotado a los 100 ms).

TABLA IV. CASOS DE PRUEBA DEL PLAN DE SIMULACIÓN

Caso	DIP Switch	Secuencia esperada de frecuencias (Hz)	Objetivo de cobertura	Cubre
1	0 0 0 0	1200 → 1200 → 1200 → 1200	Bit 0 en los 4 estados; sin flancos	S0–S3
2	1 1 1 1	2200 → 2200 → 2200 → 2200	Bit 1 en los 4 estados; sin flancos	S0–S3
3	1 0 1 0	2200 → 1200 → 2200 → 1200	Flancos alternantes en cada transición	Todas las transiciones
4	0 1 0 1	1200 → 2200 → 1200 → 2200	Complemento del caso 3	Todas las transiciones
5	1 1 0 0	2200 → 2200 → 1200 → 1200	Un único flanco a mitad de trama	Transición S1→S2
6	Sin señal (D7 desconectado)	—	Rama de tiempo de espera de pulseIn agotado	Manejo de error

VIII. RESULTADOS DE SIMULACIÓN

La Fig. 3 muestra la forma de onda teórica esperada a la salida del pin D9 (antes del filtro RC) para el caso de prueba 3 (DIP = 1,0,1,0), calculada a partir de la lógica del firmware: cada bit se sostiene durante 1000 ms y determina la frecuencia de la portadora cuadrada generada por tone().

Fig. 3. Forma de onda esperada para la secuencia DIP = 1,0,1,0

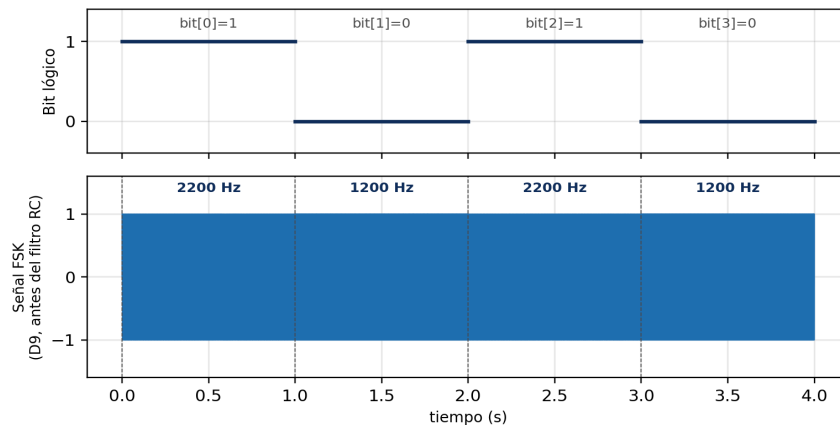


Fig. 3. Forma de onda esperada para la secuencia DIP = 1,0,1,0 (señal antes del filtro RC).

Nota metodológica: la figura anterior corresponde a la forma de onda teórica derivada del análisis del código, empleada para contrastar el comportamiento esperado con el observado. Para la entrega final de este apartado deben incorporarse las capturas de pantalla reales del monitor serial y/o del osciloscopio virtual de Tinkercad para cada caso de prueba de la Tabla IV, evidenciando el mensaje de frecuencia medida por el Arduino 2 frente al valor teórico.

TABLA V. SALIDA ESPERADA POR CONSOLA SERIAL (CASO DE PRUEBA 3)

Bit #	Bit transmitido	Frecuencia generada	Frecuencia medida (esperada)	Error esperado
1	1	2200 Hz	≈2200 Hz	< 2%
2	0	1200 Hz	≈1200 Hz	< 2%
3	1	2200 Hz	≈2200 Hz	< 2%
4	0	1200 Hz	≈1200 Hz	< 2%

IX. ANÁLISIS TEÓRICO DE LAS MÉTRICAS

A. Tasa de transmisión (baud rate). Dado que $\text{duracionBit} = 1000 \text{ ms}$, la tasa de símbolo teórica es $R_b = 1/T_{\text{bit}} = 1/1 \text{ s} = 1 \text{ bit por segundo (1 bps)}$. Esta tasa, deliberadamente baja, facilita la observación visual y la medición experimental del cambio de frecuencia.

B. Resolución de frecuencia de tone(). La función `tone()` de Arduino genera una onda cuadrada mediante un temporizador de hardware de 16 MHz. Para una frecuencia deseada f , el temporizador ajusta un divisor N tal que $f \approx 16\,000\,000 / (2 \cdot N)$, por lo que el error de cuantización relativo es inferior al 1% en el rango de audio empleado (1200–2200 Hz), resultado que se contrasta con la frecuencia medida por el Arduino 2.

C. Cálculo del error porcentual. El error relativo entre la frecuencia teórica (f_t) y la frecuencia medida (f_m) se calcula como:

$$\%error = |f_t - f_m| / f_t \times 100(1)$$

Se espera un error inferior al 5%, atribuible a la resolución del temporizador, la tolerancia de `pulseIn()` y el efecto de carga del filtro RC sobre el flanco de la señal.

D. Respuesta del filtro RC. El filtro RC de primer orden introducido a la salida del Arduino 1 tiene una frecuencia de corte teórica dada por:

$$f_c = 1 / (2\pi RC)(2)$$

donde R y C corresponden a los valores de la resistencia y el capacitor empleados en el circuito (a completar con los valores reales del montaje). Para que el filtro suavice la señal sin eliminar la información de frecuencia, f_c debe fijarse por encima de 2200 Hz, de modo que actúe como un filtro pasa-bajos suave que atenúa armónicos superiores de la onda cuadrada sin distorsionar la frecuencia fundamental transmitida.

X. DEMOSTRACIÓN EN LABORATORIO

A. Ejecución del banco de pruebas. La demostración consistirá en configurar en vivo cada combinación del DIP switch listada en la Tabla IV, observando en el monitor serial del Arduino 1 la secuencia de bits y frecuencias generadas, y

en el monitor serial del Arduino 2 la frecuencia medida correspondiente.

B. Explicación de formas de onda. Durante la demostración se señalará explícitamente el instante en que el decodificador selecciona cada dirección de la ROM de frecuencias, la transición de estado de la FSM y el acceso a memoria (arreglo `bits[]` e `indiceBit`), vinculando cada evento observado con el bloque lógico correspondiente del diagrama de la Fig. 1.

C. Preguntas anticipadas del docente. Se prevén preguntas sobre: la razón de usar `millis()` en lugar de `delay()` (no bloqueo del procesador); el motivo de la inversión lógica en la lectura del DIP switch (uso de `INPUT_PULLUP`); el efecto del filtro RC sobre la forma de onda cuadrada; y el procedimiento seguido por el Arduino 2 para diferenciar ausencia de señal de una frecuencia válida.

XI. CONCLUSIONES

El diseño detallado presentado demuestra que los conceptos de Sistemas Digitales —registro de entrada, decodificador, memoria ROM, contador, temporizador y máquina de estados finitos— pueden implementarse de forma didáctica sobre una plataforma Arduino para materializar un modulador FSK funcional de dos frecuencias. El plan de simulación propuesto garantiza cobertura de todas las transiciones de estado y de ambas ramas del decodificador, mientras que el análisis teórico de métricas provee un marco cuantitativo para validar experimentalmente el desempeño del sistema durante la demostración en laboratorio.

REFERENCIAS

- [1] Arduino, "tone() – Language Reference", Arduino Reference, Arduino AG. Disponible en: <https://www.arduino.cc/reference/en/language/functions/advanced-io/tone/>
- [2] Arduino, "pulseIn() – Language Reference", Arduino Reference, Arduino AG. Disponible en: <https://www.arduino.cc/reference/en/language/functions/advanced-io/pulsein/>
- [3] B. Sklar, Digital Communications: Fundamentals and Applications, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2001.

[4] M. M. Mano y M. D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 5th ed. Boston, MA, USA: Pearson, 2013.